



Group 1

Natalie Rose Andzie-Mensah

Nii Sowah Kwabla Sowah

Vladimir Noel Aduama

Yaa Waadu Asomaning

CS323: Database Systems/ Database Management

Supermarket Management System

Lecturer: Charles Adjetey

August 16, 2026

Table of Contents

- 1. Introduction**
- 2. Requirements analysis**
- 3. System Analysis**
- 4. Database Design and Implementation**
- 5. SQL Implementation**
- 6. Application Design and Implementation**
- 7. Testing, Security and Data Integrity**
- 8. Challenges and Solutions**
- 9. Conclusion**
- 10. References**

List of Figures:

<i>Fig 0.0</i>	<i>Fig 1.7</i>	<i>Fig 3.4</i>
<i>Fig 0.1</i>	<i>Fig 1.8</i>	<i>Fig 3.5</i>
<i>Fig 0.2</i>	<i>Fig 1.9</i>	<i>Fig 3.6</i>
<i>Fig 0.3</i>	<i>Fig 2.0</i>	<i>Fig 3.7</i>
<i>Fig 0.4</i>	<i>Fig 2.1</i>	<i>Fig 3.8</i>
<i>Fig 0.5</i>	<i>Fig 2.2</i>	<i>Fig 3.9</i>
<i>Fig 0.6</i>	<i>Fig 2.3</i>	<i>Fig 4.0</i>
<i>Fig 0.7</i>	<i>Fig 2.4</i>	<i>Fig 4.1</i>
<i>Fig 0.8</i>	<i>Fig 2.5</i>	<i>Fig 4.2</i>
<i>Fig 0.9</i>	<i>Fig 2.6</i>	<i>Fig 4.3</i>
<i>Fig 1.0</i>	<i>Fig 2.7</i>	<i>Fig 4.4</i>
<i>Fig 1.1</i>	<i>Fig 2.8</i>	<i>Fig 4.5</i>
<i>Fig 1.2</i>	<i>Fig 2.9</i>	<i>Fig 4.6</i>
<i>Fig 1.3</i>	<i>Fig 3.0</i>	<i>Fig 4.7</i>
<i>Fig 1.4</i>	<i>Fig 3.1</i>	<i>Fig 4.8</i>
<i>Fig 1.5</i>	<i>Fig 3.2</i>	<i>Fig 4.9</i>
<i>Fig 1.6</i>	<i>Fig 3.3</i>	<i>Fig 5.0</i>

List of Tables:

Table 0.0

Table 0.1

1. INTRODUCTION:

Background:

Essentials, a mini-mart located on the Ashesi University campus, is planning to expand into a full-fledged retail supermarket. To handle higher sales volumes and broader inventory, the store must replace its basic file-based records with a centralized database system. The new platform needs to track products, inventory, and supply orders, among other factors to prevent stockouts across all departments. It must also process daily customer sale transactions efficiently while managing cashier and employee operations. Additionally, Essentials wants to run targeted promotional discounts applied at checkout. Finally, an integrated loyalty program is required to award discounts to repeat customers, boosting post-expansion retention.

Problem statement:

Essentials needs a well-integrated database system that allows staff to track inventory, sales, discounts, customer and employee information.

Objectives:

Design and implement a system that:

- Tracks inventory.
- Tracks sales.
- Tracks customer and employee information.
- Tracks discounts.
- Tracks customer loyalty.

2. REQUIREMENTS ANALYSIS:

Key Stakeholders:

- Supermarket employees
- Supermarket administrators

Users:

- Cashiers
- Stock personnel
- Managers
- Administrators

Functional Requirements:

- *Product & Inventory Management*

The system shall manage product entity details, including categories, departments, unit prices, and supplier relationships. The system shall track real-time inventory levels, automatically deducting stock during sales and flagging low-stock thresholds to prevent stockouts.

- *Sales & POS Operations*

The system shall process customer checkout transactions, calculate totals with applicable pricing rules, and generate itemized receipts. The system shall record historical sales transaction entities linked to purchasing items, timestamps, and payment details.

- *Staff & User Management*

The system shall store employee entities (roles, credentials, shifts) and manage role-based permissions for system functions.

- *Discounts & Loyalty Program*

The system shall define and apply dynamic discount rules (percentage, flat rates, promotional periods) at checkout. The system shall maintain customer account entities to track purchase history, award loyalty points, and process point redemptions.

Non-Functional Requirements:

- Employee information should be stored securely and protected from unauthorized access.
- The interface shall be easy to learn for new employees.
- Users shall be able to complete common tasks with basic computer training.
- The system shall provide clear error messages and user guidance.

Assumptions:

- Purchases and customer loyalty are related.
- A loyalty discount of 15% is attracted when the total cost of items is above 1000 cedis.
- If an item expires in 45 days or less, a discount of 50% is applied.

3. SYSTEM ANALYSIS:

The current system that Essentials uses is file-based. Due to this, the employees report several accounts of missing records, as well as, complain about having to replicate certain records for different purposes. The employees also found it difficult to update the records when any change was made. This caused data inconsistency.

Upon evaluating the existing system and all its issues, we, as a team, concluded that it was best for Essentials to transition to a database system, especially since their establishment is growing.

We are proposing a management system that will allow employees to access certain records based on their roles in the company. Inventory will also be well-managed by an implemented stock alert system which indicates when stock is low. Aside from this, reports can be generated so that an overview of necessary information is easily accessed without having to visit every table to record certain details.

Business Rules

- Every purchase order must belong to a specific supplier.
- Each product belongs to a single category and has one inventory record.
- A single purchase order can include multiple products.
- Employees process customer sales, and a sale can track multiple sold products.
- Customers can optionally own one loyalty card to earn points.
- Discounts are applied per line item on a sale.

4. DATABASE DESIGN AND IMPLEMENTATION:

Database Screenshots:

ER Diagram

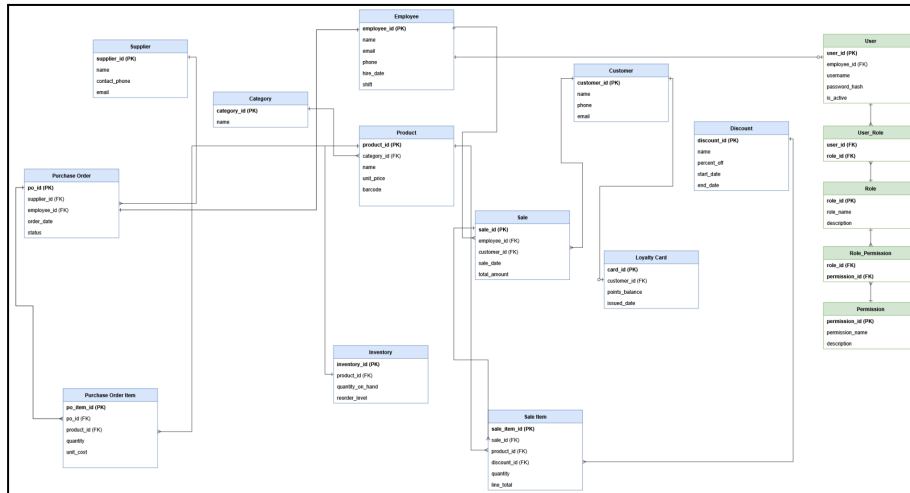


Fig 0.0

This is an entity relationship diagram which shows the entities we created for our system in terms of what the supermarket needs, and also frontend development for the web application.

Database Schema

```
MariaDB [supermarket_db]> show tables;
+-----+
| Tables_in_supermarket_db |
+-----+
| auth_group                |
| auth_group_permissions    |
| auth_permission           |
| auth_user                 |
| auth_user_groups          |
| auth_user_user_permissions|
| category                  |
| customer                  |
| discount                  |
| django_admin_log          |
| django_content_type       |
| django_migrations         |
| django_session            |
| employee                  |
| inventory                 |
| loyalty_card              |
| product                   |
| purchase_order            |
| purchase_order_item       |
| sale                      |
| sale_item                 |
| supplier                  |
| vw_cashier_daily_sales    |
| vw_low_stock_alerts       |
+-----+
```

Fig 0.1

```

MariaDB [supermarket_db]> show tables;
+-----+
| Tables_in_supermarket_db |
+-----+
| category                  |
| customer                  |
| discount                  |
| employee                  |
| inventory                 |
| loyalty_card              |
| permission                |
| product                   |
| purchase_order            |
| purchase_order_item       |
| role                      |
| role_permission           |
| sale                      |
| sale_item                 |
| supplier                  |
| user                      |
| user_role                 |
+-----+

```

Fig 0.2

This is a complete list of all tables in the database.

Table Structures

User

```

MariaDB [supermarket_db]> describe user;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| user_id        | int(11)       | NO   | PRI | NULL    | auto_increment |
| employee_id    | int(11)       | NO   | UNI | NULL    |                |
| username       | varchar(50)   | NO   | UNI | NULL    |                |
| password_hash  | varchar(255)  | NO   |     | NULL    |                |
| is_active      | tinyint(1)    | NO   |     | 1       |                |
+-----+-----+-----+-----+-----+-----+

```

Fig 0.3

This is the structure of the user table, outlining its attributes and data structures, as well as types of keys.

Role

```
MariaDB [supermarket__db]> describe role;
```

Field	Type	Null	Key	Default	Extra
role_id	int(11)	NO	PRI	NULL	auto_increment
role_name	varchar(50)	NO	UNI	NULL	
description	varchar(255)	YES		NULL	

Fig 0.4

This is the structure of the role table, outlining its attributes and data structures, as well as types of keys.

User_Role

```
MariaDB [supermarket__db]> describe user_role;
```

Field	Type	Null	Key	Default	Extra
user_id	int(11)	NO	PRI	NULL	
role_id	int(11)	NO	PRI	NULL	

Fig 0.5

This is the structure of the user_role table, outlining its attributes and data structures, as well as types of keys.

Employee

```
MariaDB [supermarket__db]> describe employee;
```

Field	Type	Null	Key	Default	Extra
employee_id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(100)	NO		NULL	
last_name	varchar(100)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
phone	varchar(20)	YES		NULL	
hire_date	date	NO		NULL	
shift	varchar(30)	YES		NULL	

Fig 0.6

This is the structure of the employee table, outlining its attributes and data structures, as well as types of keys.

Sale

```
MariaDB [supermarket__db]> describe sale;
```

Field	Type	Null	Key	Default	Extra
sale_id	int(11)	NO	PRI	NULL	auto_increment
employee_id	int(11)	NO	MUL	NULL	
customer_id	int(11)	YES	MUL	NULL	
sale_date	datetime	NO	MUL	current_timestamp()	
total_amount	decimal(10,2)	NO		0.00	

Fig 0.7

This is the structure of the sale table, outlining its attributes and data structures, as well as types of keys.

Sale_Item

```
MariaDB [supermarket__db]> describe sale_item;
```

Field	Type	Null	Key	Default	Extra
sale_item_id	int(11)	NO	PRI	NULL	auto_increment
sale_id	int(11)	NO	MUL	NULL	
product_id	int(11)	NO	MUL	NULL	
discount_id	int(11)	YES	MUL	NULL	
quantity	int(11)	NO		NULL	
line_total	decimal(10,2)	NO		NULL	

Fig 0.8

This is the structure of the sale_item table, outlining its attributes and data structures, as well as types of keys.

Loyalty_Card

```
MariaDB [supermarket_db]> describe loyalty_card;
```

Field	Type	Null	Key	Default	Extra
card_id	int(11)	NO	PRI	NULL	auto_increment
customer_id	int(11)	NO	UNI	NULL	
points_balance	int(11)	NO		0	
issued_date	date	NO		curdate()	

Fig 0.9

This is the structure of the loyalty_card table, outlining its attributes and data structures, as well as types of keys.

Supplier

```
MariaDB [supermarket_db]> describe supplier;
```

Field	Type	Null	Key	Default	Extra
supplier_id	int(11)	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	
contact_phone	varchar(20)	YES		NULL	
email	varchar(100)	YES	UNI	NULL	

Fig 1.0

This is the structure of the supplier table, outlining its attributes and data structures, as well as types of keys.

Primary and Foreign Keys

Table Name	Primary Key	Foreign Key(s)
SUPPLIER	supplier_id	—
CATEGORY	category_id	—
EMPLOYEE	employee_id	—
PURCHASE ORDER	po_id	supplier_id, employee_id
PURCHASE ORDER ITEM	po_item_id	po_id, product_id
PRODUCT	product_id	category_id
INVENTORY	inventory_id	product_id
CUSTOMER	customer_id	—
LOYALTY CARD	card_id	customer_id
SALE	sale_id	employee_id, customer_id
SALE ITEM	sale_item_id	Sale_id, product_id, discount_id

DISCOUNT	discount_id	—
USER	user_id	employee_id
USER ROLE	—	user_id, role_id
ROLE	role_id	—
ROLE PERMISSION	—	role_id, permission_id
PERMISSION	permission_id	—

Table 0.0

This table describes the primary and foreign keys in each table.

RECORDS

User

```
MariaDB [supermarket__db]> select * from user;
```

user_id	employee_id	username	password_hash	is_active
1	1	kwesi	\$2b\$12\$eImiTXuWVxfM37uY4JANj0Qe3u3E4/y24k1.2k33	1
2	2	ama	\$2b\$12\$eImiTXuWVxfM37uY4JANj0Qe3u3E4/y24k1.2k34	1
3	3	yaw	\$2b\$12\$eImiTXuWVxfM37uY4JANj0Qe3u3E4/y24k1.2k35	1
4	4	efia	\$2b\$12\$eImiTXuWVxfM37uY4JANj0Qe3u3E4/y24k1.2k36	1
5	5	kwame	\$2b\$12\$eImiTXuWVxfM37uY4JANj0Qe3u3E4/y24k1.2k37	1

Fig 1.1

Role

role_id	role_name	description
1	Store Manager	Full operational and inventory oversight
2	Head Cashier	Supervisory POS and terminal manager
3	Cashier	Standard POS terminal checkout staff
4	Inventory Officer	Stock receiving and warehouse management

Fig 1.2

Customer

MariaDB [supermarket_db]> select * from customer;

customer_id	first_name	last_name	phone	email
1	Kofi	Agyemang	0244100001	kofi.agyemang@gmail.com
2	Abena	Dankwa	0208200002	abena.dankwa@yahoo.com
3	Kwabena	Frimpong	0553300003	k.frimpong@outlook.com
4	Akosua	Kyei	0245400004	akosua.kyei@gmail.com
5	Kojo	Antwi	0501500005	kojo.antwi@hotmail.com
6	Yaa	Asantewaa	0246600006	yaa.asantewaa@gmail.com
7	Kwaku	Sarpong	0277700007	kwaku.sarpong@yahoo.com
8	Afia	Acheampong	0208800008	afia.ach@gmail.com
9	Mawuli	Dogbe	0509900009	mawuli.dogbe@gmail.com
10	Selorm	Gbeku	0241010010	selorm.gbeku@yahoo.com
11	Kekeli	Avoke	0502120011	kekeli.avoke@gmail.com
12	Esi	Quaye	0273230012	esi.quaye@hotmail.com
13	Nii	Armah	0244340013	nii.armah@gmail.com
14	Naa	Lartey	0205450014	naa.lartey@yahoo.com
15	Kpakpo	Allotey	0556560015	kpakpo.allotey@gmail.com
16	Fuseini	Iddrisu	0247670016	fuseini.idrisu@gmail.com
17	Aisha	Mohammed	0508780017	aisha.m@yahoo.com
18	Ibrahim	Yakubu	0279890018	ibrahim.yakubu@hotmail.com
19	Aminu	Bawa	0241001019	aminu.bawa@gmail.com
20	Fatima	Alhassan	0202112020	fatima.alhassan@yahoo.com
21	Kobina	Eshun	0553223021	kobina.eshun@gmail.com
22	Araba	Forson	0244334022	araba.forson@gmail.com
23	Ekow	Baidoo	0505445023	ekow.baidoo@hotmail.com
24	Adwoa	Sackey	0276556024	adwoa.sackey@yahoo.com
25	Ato	Crentsil	0247667025	ato.crentsil@gmail.com

Fig 1.3

Employee

MariaDB [supermarket_db]> select * from employee;

employee_id	first_name	last_name	email	phone	hire_date	shift
1	Kwesi	Mensah	kwesi.mensah@supermarket.gh	0244111222	2024-01-10	Morning
2	Ama	Osei	ama.osei@supermarket.gh	0208333444	2024-02-15	Morning
3	Yaw	Addo	yaw.addo@supermarket.gh	0553555666	2024-05-01	Evening
4	Efia	Appiah	efia.appiah@supermarket.gh	0245777888	2024-06-20	Morning
5	Kwame	Boateng	kwame.boateng@supermarket.gh	0501999000	2025-01-10	Evening

Fig 1.4

Product

```
MariaDB [supermarket_db]> select * from product;
```

product_id	category_id	name	unit_price	barcode
1	1	Ideal Milk 160g Tin	8.50	GH0001001
2	1	Milo 400g Tin	38.00	GH0001002
3	1	FanYogo Strawberry 145ml	4.00	GH0001003
4	1	FanChoco 145ml	4.00	GH0001004
5	3	Gino Tomato Paste 70g Sachet	3.50	GH0001005
6	3	Frytol Vegetable Oil 1L	45.00	GH0001006
7	3	Indomie Onion Chicken 70g	3.00	GH0001007
8	6	Club Premium Lager 625ml	14.00	GH0001008
9	6	Eagle Extra Stout 500ml	12.00	GH0001009
10	6	Alomo Bitters 750ml	35.00	GH0001010
11	2	Golden Tree Kingsbite Chocolate 50g	12.00	GH0001011
12	1	Nescafé 3-in-1 Classic Sachet	2.50	GH0001012
13	1	Cowbell Milk Powder 400g Sachet	28.00	GH0001013
14	1	YumVita Infant Cereal 250g	22.00	GH0001014
15	1	Bel-Aqua Mineral Water 1.5L	5.00	GH0001015
16	1	Awake Purified Water 500ml	2.50	GH0001016
17	3	Geisha Mackerel Tomato Sauce 425g	24.00	GH0001017
18	3	Royco Beef Seasoning Cubes (50 Pack)	18.00	GH0001018
19	3	Joff Rice Fragrant Rice 5kg	110.00	GH0001019
20	3	Key Soap 800g Bar	22.00	GH0001020
21	4	Geisha Aloe Vera Soap 150g	8.00	GH0001021
22	4	Pepsodent Cavity Fighter 175g	16.00	GH0001022
23	5	Omo Multi-Active Washing Powder 500g	18.00	GH0001023
24	5	Sunlight Dishwashing Liquid 750ml	25.00	GH0001024
25	1	Malta Guinness 330ml Can	9.00	GH0001025
26	1	Alvaro Pear Drink 330ml Bottle	8.50	GH0001026
27	1	Bel-Cola 500ml	4.50	GH0001027
28	1	Squeeze Orange Juice 1L	18.00	GH0001028
29	1	Beta Malt 330ml Pet Bottle	8.00	GH0001029
30	2	Miksi Chocolate Powder 30g Sachet	2.00	GH0001030
31	2	Golden Tree Akuafo Bar 50g	12.00	GH0001031
32	2	FanIce Vanilla 145ml	5.00	GH0001032
33	1	Bel-Chill Mango Juice 330ml	4.00	GH0001033
34	2	Loya Milk Powder 30g Sachet	2.20	GH0001034
35	1	Cerelac Wheat 400g Tin	42.00	GH0001035

Fig 1.5

5. SQL IMPLEMENTATION:

Views

```
Database > SQL
-- View 1: Low Stock Alerts
CREATE OR REPLACE VIEW vw_low_stock_alerts AS
SELECT
  p.product_id,
  p.name AS product_name,
  c.name AS category_name,
  i.quantity_on_hand,
  i.reorder_level,
  (i.reorder_level - i.quantity_on_hand) AS shortage_amount,
  CASE
    WHEN i.quantity_on_hand <= 0 THEN 'Out of Stock'
    ELSE 'Reorder Required'
  END AS alert_status
FROM inventory i
JOIN product p ON i.product_id = p.product_id
JOIN category c ON p.category_id = c.category_id
WHERE i.quantity_on_hand <= i.reorder_level;

-- View 2: Daily Cashier Performance
CREATE OR REPLACE VIEW vw_cashier_daily_sales AS
SELECT
  e.employee_id,
  e.first_name,
  e.last_name,
  CONCAT(e.first_name, ' ', e.last_name) AS cashier_name,
  DATE(s.sale_date) AS sales_date,
  COUNT(s.transaction_id) AS total_transactions,
  COALESCE(SUM(s.total_amount), 0) AS total_revenue
FROM employee e
JOIN sale s ON e.employee_id = s.employee_id
GROUP BY e.employee_id, e.first_name, e.last_name, DATE(s.sale_date)
ORDER BY e.last_name, e.first_name;

-- View 3: Customer Loyalty Summary
CREATE OR REPLACE VIEW vw_customer_summary AS
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  c.phone,
  c.email,
  COALESCE(lc.points_balance, 0) AS points_balance,
  COUNT(s.transaction_id) AS total_purchases,
  COALESCE(SUM(s.total_amount), 0) AS total_spent
FROM customer c
LEFT JOIN loyalty_card lc ON c.customer_id = lc.customer_id
LEFT JOIN sale s ON c.customer_id = s.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.phone, c.email, lc.points_balance
ORDER BY c.last_name, c.first_name;
```

Fig 1.6

Creates an overview of information of different entities.

Queries

```
Database > SQL
-- 1: Rank products by sales revenue within each category using DENSE_RANK()
SELECT
  c.name AS category_name,
  p.name AS product_name,
  SUM(s.line_total) AS total_revenue,
  DENSE_RANK() OVER (PARTITION BY c.category_id ORDER BY SUM(s.line_total) DESC) AS revenue_rank
FROM product p
JOIN category c ON p.category_id = c.category_id
JOIN sale_item si ON p.product_id = si.product_id
JOIN sale s ON si.sale_id = s.sale_id
GROUP BY c.category_id, c.name, p.product_id, p.name;

-- 2: Identify high-value customers who spend more than the average customer spending
WITH CustomerSpending AS (
  SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(s.total_amount) AS total_spent
  FROM customer c
  JOIN sale s ON c.customer_id = s.customer_id
  GROUP BY c.customer_id, c.first_name, c.last_name
)
SELECT
  first_name,
  last_name,
  total_spent
FROM CustomerSpending
WHERE total_spent > (SELECT AVG(total_spent) FROM CustomerSpending)
ORDER BY last_name, first_name;

-- 3: Running total of daily sales revenue using window functions
SELECT
  DATE(sale_date) AS sales_date,
  SUM(total_amount) AS daily_revenue,
  SUM(SUM(total_amount) OVER (ORDER BY DATE(sale_date))) AS cumulative_running_revenue
FROM sale
GROUP BY DATE(sale_date);

-- 4: Month-over-month revenue comparison using LAG()
WITH MonthlyRevenue AS (
  SELECT
    DATE_FORMAT(sale_date, 'Y-%m') AS sales_month,
    SUM(total_amount) AS current_month_revenue
  FROM sale
  GROUP BY DATE_FORMAT(sale_date, 'Y-%m')
)
SELECT
  sales_month,
  current_month_revenue,
  LAG(current_month_revenue, 1) OVER (ORDER BY sales_month) AS previous_month_revenue,
  ROUND((current_month_revenue - LAG(current_month_revenue, 1) OVER (ORDER BY sales_month)) /
    LAG(current_month_revenue, 1) OVER (ORDER BY sales_month) * 100, 2) AS moa_growth_percent
FROM MonthlyRevenue;
```

Fig 1.7

Utilizes different sql functions and joins to perform system specific tasks.

Triggers

```
1  -- Connect to MSSQL | Run on same connection | = Select block | FutureProofDev, 10 minutes ago | I author (FutureProofDev)
2  USE supermarket_db;
3
4  -- AUTOMATED TRIGGERS
5
6  DELIMITER //
7
8
9
10 -- Trigger 2: Auto-add loyalty points on completed checkout
11 CREATE TRIGGER trg_after_sale_loyalty_update
12 AFTER INSERT ON sale
13 FOR EACH ROW
14 BEGIN
15     IF NEW.customer_id IS NOT NULL THEN
16         UPDATE loyalty_card
17         SET points_balance = points_balance + FLOOR(NEW.total_amount / 10)
18         WHERE customer_id = NEW.customer_id;
19     END IF;
20 END//
21
22 -- Trigger 3: Prevent sale item insert if stock on hand is not enough
23 CREATE TRIGGER trg_prevent_insufficient_stock
24 BEFORE INSERT ON sale_item
25 FOR EACH ROW
26 BEGIN
27     DECLARE v_available_qty INT;
28
29     SELECT quantity_on_hand INTO v_available_qty
30     FROM inventory
31     WHERE product_id = NEW.product_id;
32
33     IF v_available_qty < NEW.quantity THEN
34         SIGNAL SQLSTATE '45000'
35         SET MESSAGE_TEXT = 'Transaction aborted: Insufficient stock on hand for this product.';
36     END IF;
37 END//
38
39 DELIMITER ;
```

Fig 1.8 and Fig 1.9

Waits on certain database operations to occur and perform a corresponding action.

Procedures

```
54 LEFT JOIN sale s ON c.customer_id = s.customer_id
55 WHERE c.customer_id = p_customer_id
56 GROUP BY c.customer_id, c.first_name, c.last_name, c.email, c.phone_number, c.created_at;
57
58 -- Recent transactions
59 SELECT
60     sale_id,
61     sale_date,
62     total_amount,
63     payment_method
64 FROM sale
65 WHERE customer_id = p_customer_id
66 ORDER BY sale_date DESC
67 LIMIT 5;
68 END //
69
70 DELIMITER ;
71
72 DELIMITER //
73
74 DROP PROCEDURE IF EXISTS sp_customer_order_history //
75
76 CREATE PROCEDURE sp_customer_order_history(IN p_customer_id INT)
77 BEGIN
78     SELECT
79         s.id AS sale_id,
80         s.created_at AS sale_date,
81         s.total_amount,
82         s.payment_method
83     FROM sale s
84     WHERE s.customer_id = p_customer_id
85     ORDER BY s.created_at DESC;
86 END //
87
88 DELIMITER ;
```

```
54 LEFT JOIN sale s ON c.customer_id = s.customer_id
55 WHERE c.customer_id = p_customer_id
56 GROUP BY c.customer_id, c.first_name, c.last_name, c.email, c.phone_number, c.created_at;
57
58 -- Recent transactions
59 SELECT
60     sale_id,
61     sale_date,
62     total_amount,
63     payment_method
64 FROM sale
65 WHERE customer_id = p_customer_id
66 ORDER BY sale_date DESC
67 LIMIT 5;
68 END //
69
70 DELIMITER ;
71
72 DELIMITER //
73
74 DROP PROCEDURE IF EXISTS sp_customer_order_history //
75
76 CREATE PROCEDURE sp_customer_order_history(IN p_customer_id INT)
77 BEGIN
78     SELECT
79         s.id AS sale_id,
80         s.created_at AS sale_date,
81         s.total_amount,
82         s.payment_method
83     FROM sale s
84     WHERE s.customer_id = p_customer_id
85     ORDER BY s.created_at DESC;
86 END //
87
88 DELIMITER ;
```

Fig 2.0

A group of sql operations that run sequentially.

6. APPLICATION DESIGN AND IMPLEMENTATION

Login Page

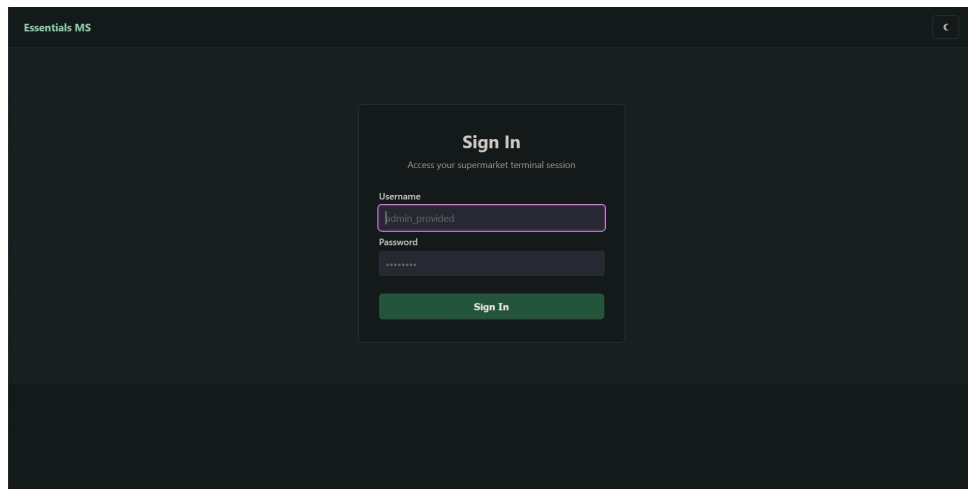


Fig 2.1

This is an image of the login page which allows access to a user based on their role in the supermarket.

Main Dashboard

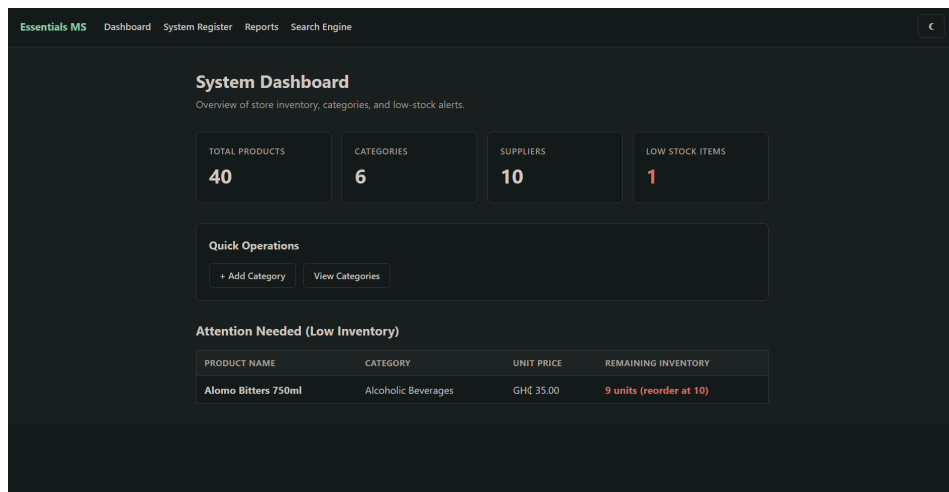


Fig 2.2

This is an image of the main dashboard page which shows general information of the supermarket once logged in.

Main Data Entry Form

Essentials MS

Dashboard

System Register

Reports

Search Engine

Add Product

Category:

Dairy & Beverages

Supplier:

Nestlé Ghana Limited

Name:

Ideal Milk

Unit price:

1

Barcode:

Expiry date:

mm / dd / yyyy

Save

Back

Fig 2.3

This page is for adding products to inventory.

Record Listing

Essentials MS

Dashboard

System Register

Reports

Search Engine

Products

+ Add Product

NAME	CATEGORY	SUPPLIER	PRICE	EXPIRY	
Alomo Bitters 750ml	Alcoholic Beverages	Kasapreko Company Limited	GHS 35.00	—	View
Alvaro Pear Drink 330ml Bottle	Dairy & Beverages	Guinness Ghana Breweries PLC	GHS 8.50	—	View
Awake Purified Water 500ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 2.50	—	View
Bel-Aqua Mineral Water 1.5L	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 5.00	—	View
Bel-Chill Mango Juice 330ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 4.00	—	View
Bel-Cola 500ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 4.50	—	View
Beta Malt 330ml Pet Bottle	Dairy & Beverages	Accra Brewery Limited	GHS 8.00	—	View
Cerelec Wheat 400g Tin	Dairy & Beverages	Nestlé Ghana Limited	GHS 42.00	—	View
...			GHS		

Fig 2.4

This page shows a list of the products available.

Search or Filtering Functionality

Essentials MS

Dashboard

System Register

Reports

Search Engine

Dynamic Search & Query Builder

Search across store entities, filter relational dependencies, and sort result sets.

TARGET ENTITY

Products Catalog

KEYWORD SEARCH

Search name, barcode, ID...

CATEGORY

Dairy & Beverages

SUPPLIER

Accra Brewery Limited

SORT PROPERTY

Product Name

SORT ORDER

Ascending (↑)

Apply Filter Query

Reset Query

Query Results

1 Record(s) Found

BARCODE	PRODUCT NAME	CATEGORY	SUPPLIER	PRICE	EXPIRY
GH0001829	Beta Malt 330ml Pet Bottle	Dairy & Beverages	Accra Brewery Limited	GHS 8.00	—

Fig 2.5

This allows a user to locate a certain item or group of items in the database.

Update Functionality

Essentials MS

Dashboard

System Register

Reports

Search Engine

Edit Product

Category:

Alcoholic Beverages

Supplier:

Kasapreko Company Limited

Name:

Alomo Bitters 750ml

Unit price:

35.00

Barcode:

GH0001010

Expiry date:

dd/mm/yyyy

Save

Back

Fig 2.6

This page allows a user to change the information of a particular item in the database.

Delete/Deactivation Functionality

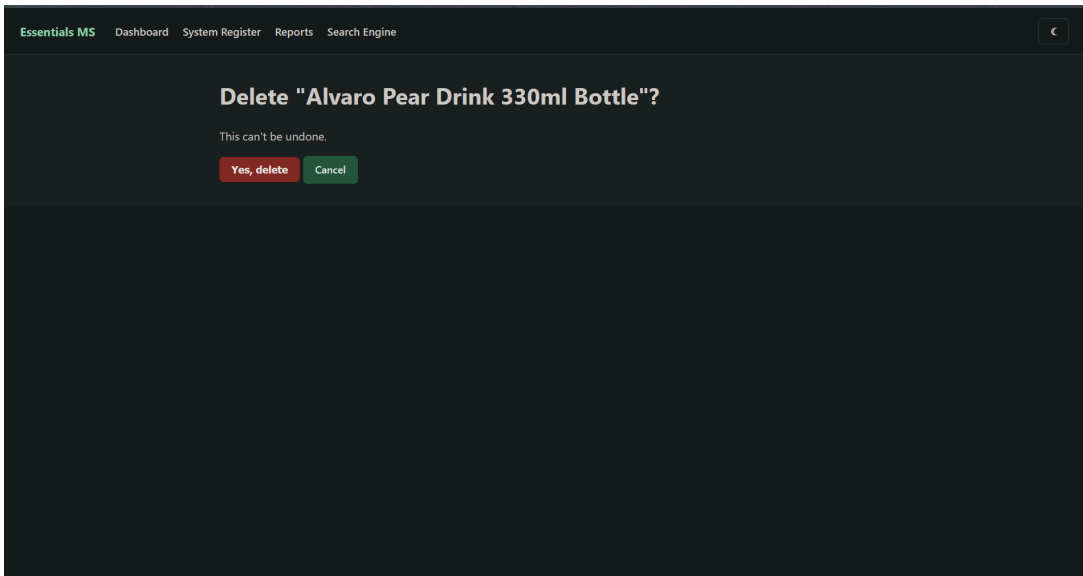


Fig 2.7

This allows a user to remove an item from inventory.

Important Transaction Screen

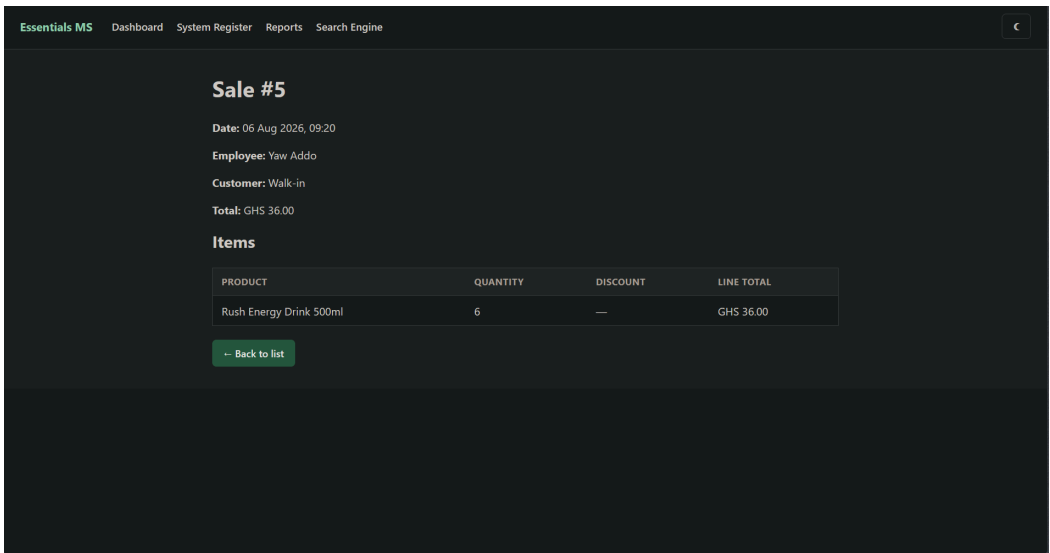


Fig 2.8

This shows the summary of a particular transaction.

Report Generation

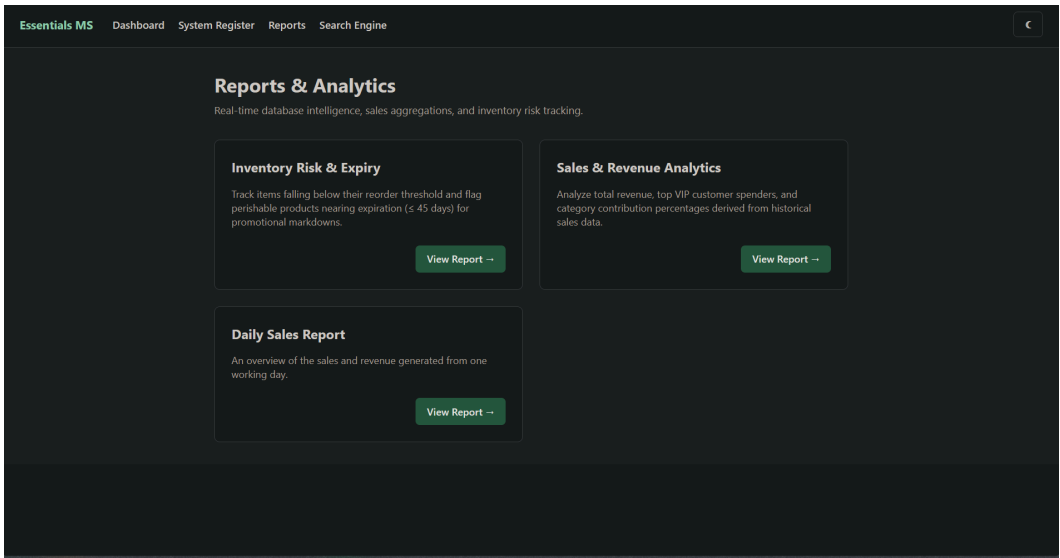


Fig 2.9

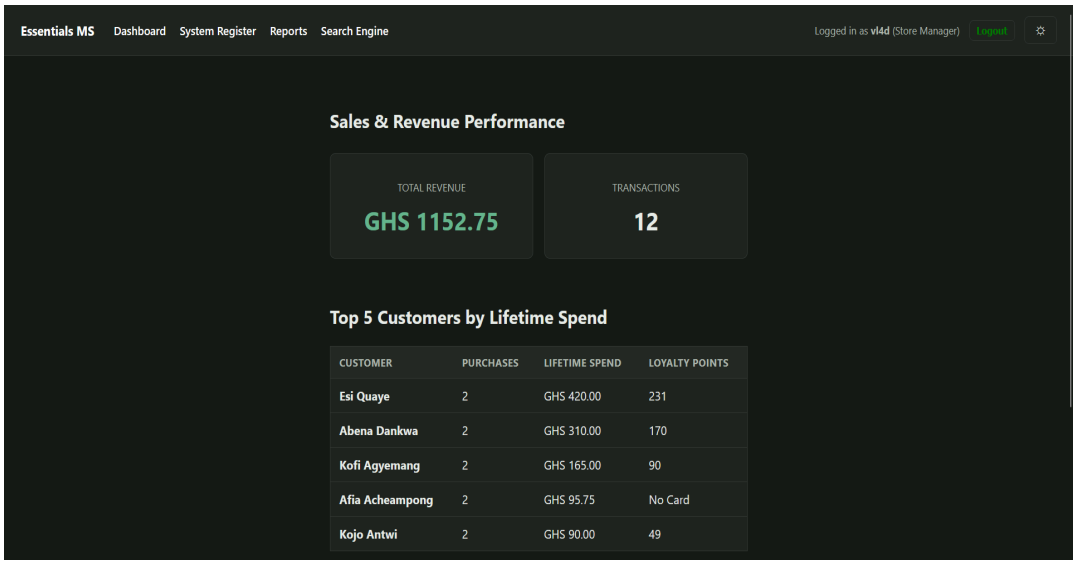
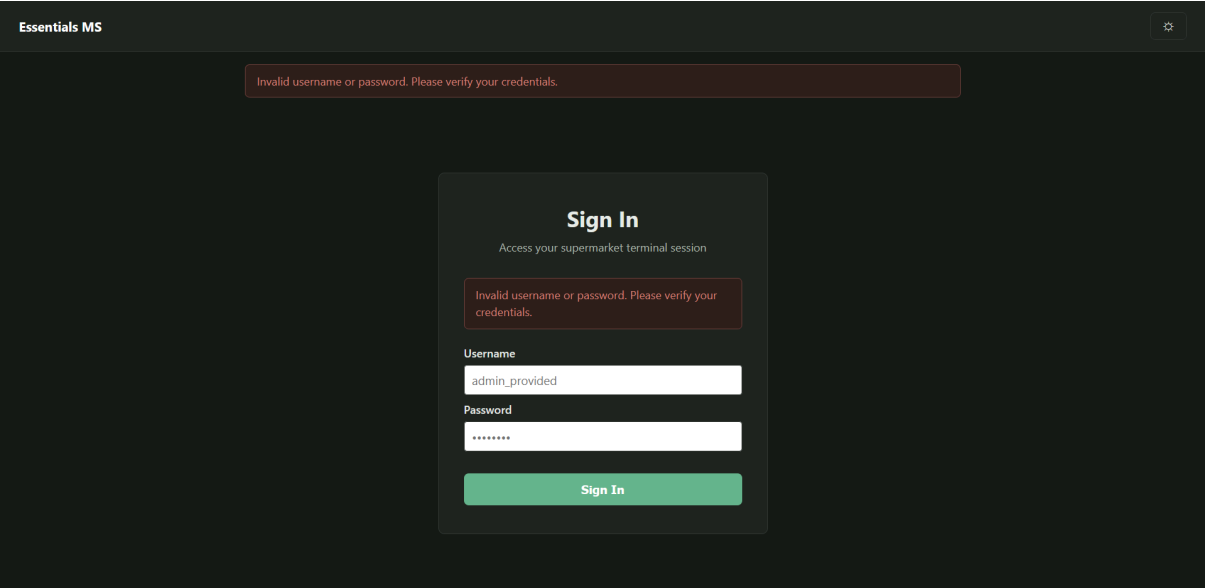


Fig 3.0

This shows the reports page and shows the report for general sales and revenue performance.

Error or Validation Handling

Incorrect password

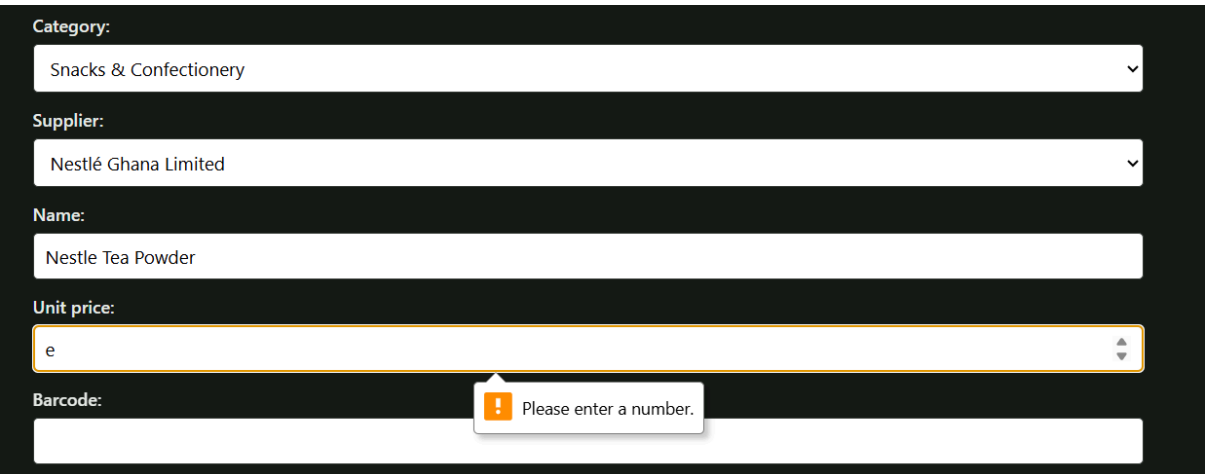


The screenshot shows a web application interface for 'Essentials MS'. At the top, there is a dark header bar with the text 'Essentials MS' on the left and a settings icon on the right. Below the header, a dark background contains a central 'Sign In' form. The form has a title 'Sign In' and a subtitle 'Access your supermarket terminal session'. It includes two input fields: 'Username' with the value 'admin_provided' and 'Password' with masked characters '*****'. A green 'Sign In' button is at the bottom of the form. Above the form, there is a red error message box that reads 'Invalid username or password. Please verify your credentials.'.

Fig 3.1

This shows that a user will not be granted access if the password or username they enter is incorrect.

User Input:



The screenshot shows a form with several fields. The 'Category' field is a dropdown menu with 'Snacks & Confectionery' selected. The 'Supplier' field is a dropdown menu with 'Nestlé Ghana Limited' selected. The 'Name' field is a text input with 'Nestle Tea Powder'. The 'Unit price' field is a text input with 'e'. The 'Barcode' field is a text input. A red error message box with an exclamation mark icon is displayed over the 'Barcode' field, reading 'Please enter a number.'.

Fig 3.2

This shows that if a user enters data of a certain type in a certain field which does not accept that data type, it is flagged and an error message is displayed.

User/Role Management

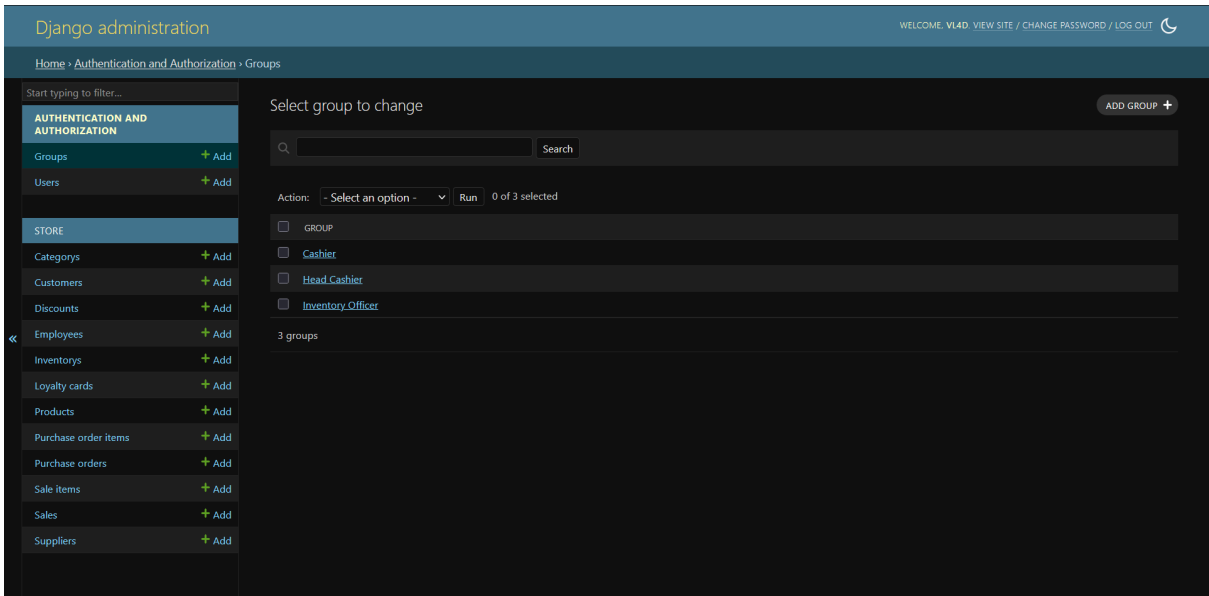


Fig 3.3

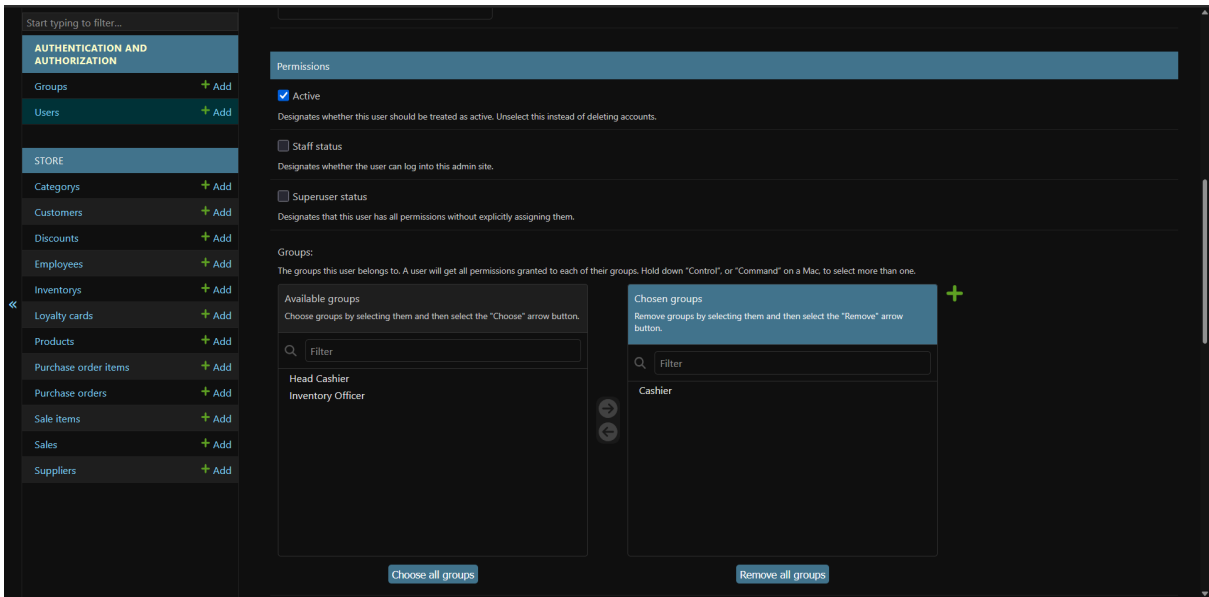


Fig 3.4

It allows the shop manager to assign roles.

Evidence of Database - Application Integration

Initial product list with no blue band.

NAME	CATEGORY	SUPPLIER	PRICE	EXPIRY	
Alomo Bitters 750ml	Alcoholic Beverages	Kasapreko Company Limited	GHS 35.00	—	View
Alvaro Pear Drink 330ml Bottle	Dairy & Beverages	Guinness Ghana Breweries PLC	GHS 8.50	—	View
Awake Purified Water 500ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 2.50	—	View
Bel-Aqua Mineral Water 1.5L	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 5.00	—	View
Bel-Chill Mango Juice 330ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 4.00	—	View
Bel-Cola 500ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 4.50	—	View
Beta Malt 330ml Pet Bottle	Dairy & Beverages	Accra Brewery Limited	GHS 8.00	—	View
Cerelac Wheat 400g Tin	Dairy & Beverages	Nestlé Ghana Limited	GHS 42.00	—	View

Fig 3.5

Corresponding table in db.

```
MariaDB [supermarket_db]> select * from product order by name;
```

product_id	name	unit_price	barcode	expiry_date	category_id	supplier_id
10	Alomo Bitters 750ml	35.00	GH0001010	NULL	6	4
26	Alvaro Pear Drink 330ml Bottle	8.50	GH0001026	NULL	1	9
16	Awake Purified Water 500ml	2.50	GH0001016	NULL	1	7
15	Bel-Aqua Mineral Water 1.5L	5.00	GH0001015	NULL	1	7
33	Bel-Chill Mango Juice 330ml	4.00	GH0001033	NULL	1	7
27	Bel-Cola 500ml	4.50	GH0001027	NULL	1	7
29	Beta Malt 330ml Pet Bottle	8.00	GH0001029	NULL	1	8
35	Cerelac Wheat 400g Tin	42.00	GH0001035	NULL	1	2
8	Club Premium Lager 625ml	14.00	GH0001008	NULL	6	8
13	Cowbell Milk Powder 400g Sachet	28.00	GH0001013	NULL	1	6
9	Eagle Extra Stout 500ml	12.00	GH0001009	NULL	6	8

Fig 3.6

User record creation:

Essentials MS

Dashboard

System Register

Reports

Search Engine

Logged in as v14d (Store Manager)

Logout

Add Product

Category:

Groceries & Staples

Supplier:

Nestlé Ghana Limited

Name:

Blue Band Margarin

Unit price:

43.5

Barcode:

GH012345664343

Expiry date:

01 / 01 / 2029

Save

Back

Fig 3.7

Product table after record creation:

MariaDB [supermarket_db]> select * from product order by name;

product_id	name	unit_price	barcode	expiry_date	category_id	supplier_id
10	Alomo Bitters 750ml	35.00	GH0001010	NULL	6	4
26	Alvaro Pear Drink 330ml Bottle	8.50	GH0001026	NULL	1	9
16	Awake Purified Water 500ml	2.50	GH0001016	NULL	1	7
15	Bel-Aqua Mineral Water 1.5L	5.00	GH0001015	NULL	1	7
33	Bel-Chill Mango Juice 330ml	4.00	GH0001033	NULL	1	7
27	Bel-Cola 500ml	4.50	GH0001027	NULL	1	7
29	Beta Malt 330ml Pet Bottle	8.00	GH0001029	NULL	1	8
83	Blue Band Margarin	43.50	GH012345664343	2029-01-01	3	2
35	Cerelac Wheat 400g Tin	42.00	GH0001035	NULL	1	2
8	Club Premium Lager 625ml	14.00	GH0001008	NULL	6	8

Fig 3.8

Application:

Essentials MS

DashboardSystem RegisterReportsSearch Engine

Logged in as v14d (Store Manager)Logout

Products

+ Add Product

NAME	CATEGORY	SUPPLIER	PRICE	EXPIRY	
Alomo Bitters 750ml	Alcoholic Beverages	Kasapreko Company Limited	GHS 35.00	—	View
Alvaro Pear Drink 330ml Bottle	Dairy & Beverages	Guinness Ghana Breweries PLC	GHS 8.50	—	View
Awake Purified Water 500ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 2.50	—	View
Bel-Aqua Mineral Water 1.5L	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 5.00	—	View
Bel-Chill Mango Juice 330ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 4.00	—	View
Bel-Cola 500ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 4.50	—	View
Beta Malt 330ml Pet Bottle	Dairy & Beverages	Accra Brewery Limited	GHS 8.00	—	View
Blue Band Margarin	Groceries & Staples	Nestlé Ghana Limited	GHS 43.50	Jan. 1, 2029	View

Fig 3.9

7. TESTING, SECURITY AND DATA INTEGRITY

Table 0.1

Test ID	Feature	Test Input	Expected Result	Actual Result	Status
T01	User Authentication	Valid credentials (kyesi / password123)	User successfully authenticated; redirected to dashboard	User logged in to terminal session	Pass
T02	Login Validation	Invalid password for existing user	Access rejected; error message displayed	Error banner displayed; session blocked	Pass
T03	Product Catalog	Valid product data (Milo 400g, GH0001002, GHS 38.00)	New product record saved to database	Product saved and visible in catalog	Pass
T04	Uniqueness Constraint	Duplicate barcode (GH0001010)	Database unique constraint	Duplicate error displayed;	Pass

			rejects insertion	record rejected	
T05	Dynamic Search Engine	Category filter = Dairy & Beverages	Matching subset of products returned	Filtered product records returned	Pass
T06	Foreign Key Deletion Protection	Delete category with linked products	ProtectedError r caught; deletion aborted	Error alert shown; integrity maintained	Pass
T07	POS Inventory Deduction	Sale checkout for 2 units of Ideal Milk	Sale recorded; inventory quantity decremented by 2	Sale finalized; stock updated from 148 to 143	Pass

T01- User Authentication

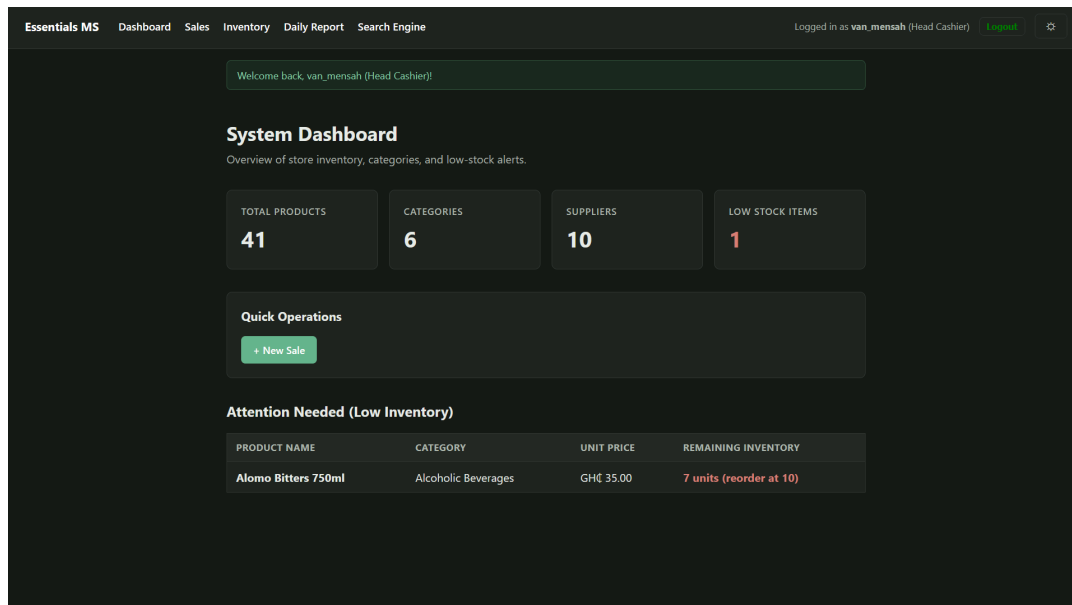


Fig 4.0

*This shows a valid login when a user enters valid credentials. This is proven by the **welcome** message at the top of the screen.*

T02 - Login Validation

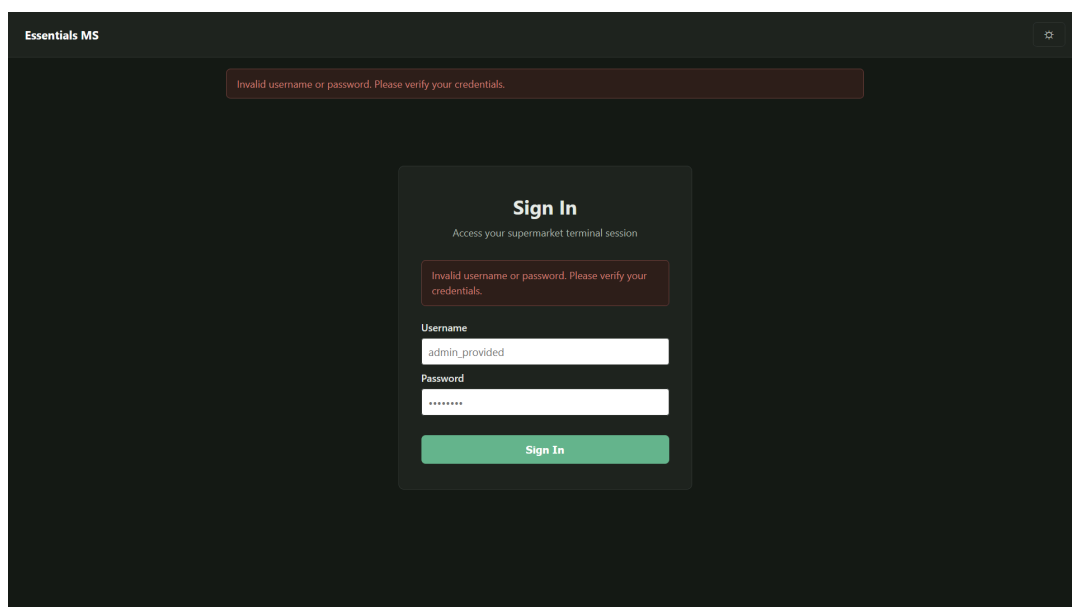


Fig 4.1

This throws an error because a user has entered invalid credentials.

T03 - Product Catalog

Essentials MS

Dashboard

System Register

Reports

Search Engine

Logged in as v14d (Store Manager)

Logout

Add Product

Category:

Groceries & Staples

Supplier:

Nestlé Ghana Limited

Name:

Milo 400g

Unit price:

38.0

Barcode:

GH0001002

Expiry date:

03 / 08 / 2027

Save

Back

Fig 4.2

Confirmation Page

Essentials MS

Dashboard

System Register

Reports

Search Engine

Logged in as v14d (Store Manager)

Logout

Product "Milo 400g" created.

Milo 400g

Category:

Groceries & Staples

Supplier:

Nestlé Ghana Limited

Unit price:

GHS 38.00

Barcode:

GH0001234

Expiry date:

March 8, 2027

Inventory:

No inventory record

Edit

Delete

Back to list

Fig 4.3

This shows a user creating and adding an item to the inventory, and a confirmation page to prove that the item has been added to inventory.

T04 - Uniqueness Constraint

The screenshot displays the 'Add Product' interface within the 'Essentials MS' application. The top navigation bar includes links to 'Dashboard', 'System Register', 'Reports', and 'Search Engine', along with a user status indicator 'Logged in as v14d (Store Manager)' and a 'Logout' button. The main form area is titled 'Add Product' and contains the following fields:

- Category:** A dropdown menu currently showing 'Snacks & Confectionery'.
- Supplier:** A dropdown menu currently showing 'Nestlé Ghana Limited'.
- Name:** A text input field containing 'Mars Bars'.
- Unit price:** A text input field containing '14.5'.
- Barcode:** A text input field containing 'GH0001010'.
- Expiry date:** A date picker field showing '03 / 31 / 2029'.

Below the 'Unit price' field, a red error message is displayed: 'Product with this Barcode already exists.' At the bottom of the form, there are two buttons: a large green 'Save' button and a smaller green 'Back' button.

Fig 4.4

This shows that an item with the entered barcode already exists, proving uniqueness of the keys in the database and error handling.

T05 - Dynamic Search Engine

Dynamic Search & Query Builder

Search across store entities, filter relational dependencies, and sort result sets.

TARGET ENTITY

Products Catalog

KEYWORD SEARCH

Search name, barcode, ID...

CATEGORY

-- All Categories --

SUPPLIER

-- All Suppliers --

SORT PROPERTY

Product Name

SORT ORDER

Ascending (↑)

Apply Filter Query

Reset Query

Query Results

42 Record(s) Found

BARCODE	PRODUCT NAME	CATEGORY	SUPPLIER	PRICE	EXPIRY
GH0001010	Alomo Bitters 750ml	Alcoholic Beverages	Kasapreko Company Limited	GHS 35.00	—
GH0001026	Alvaro Pear Drink 330ml Bottle	Dairy & Beverages	Guinness Ghana Breweries PLC	GHS 8.50	—
GH0001016	Awake Purified Water 500ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 2.50	—
GH0001015	Bel-Aqua Mineral Water 1.5L	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 5.00	—
GH0001033	Bel-Chill Mango Juice 330ml	Dairy & Beverages	Blow Chem Industries (Bel Aqua)	GHS 4.00	—
			Blow Chem Industries		

Fig 4.5

Essentials MS

Dashboard

System Register

Reports

Search Engine

Logged in as v14d (Store Manager)

Logout

Dynamic Search & Query Builder

Search across store entities, filter relational dependencies, and sort result sets.

TARGET ENTITY

Products Catalog

KEYWORD SEARCH

Search name, barcode, ID...

CATEGORY

Personal Care & Hygiene

SUPPLIER

-- All Suppliers --

SORT PROPERTY

Product Name

SORT ORDER

Ascending (↑)

Apply Filter Query

Reset Query

Query Results

2 Record(s) Found

BARCODE	PRODUCT NAME	CATEGORY	SUPPLIER	PRICE	EXPIRY
GH0001021	Geisha Aloe Vera Soap 150g	Personal Care & Hygiene	Unilever Ghana PLC	GHS 8.00	—
GH0001022	Pepsodent Cavity Fighter 175g	Personal Care & Hygiene	Unilever Ghana PLC	GHS 16.00	—

Fig 4.6

Allows the user to apply multiple filters to narrow down their search.

T06 - Foreign Key Constraint

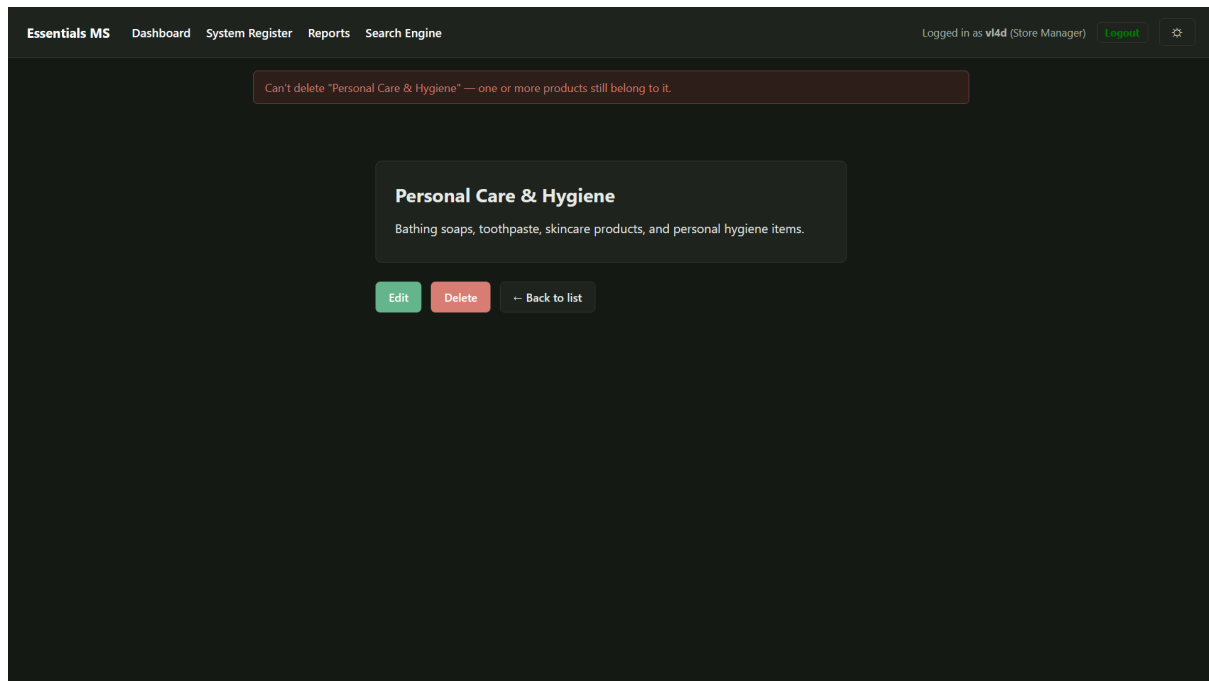


Fig 4.7

It handles reference integrity constraint by preventing deletion of an entity which is referenced by others.

T07 - Automatic Stock Deduction

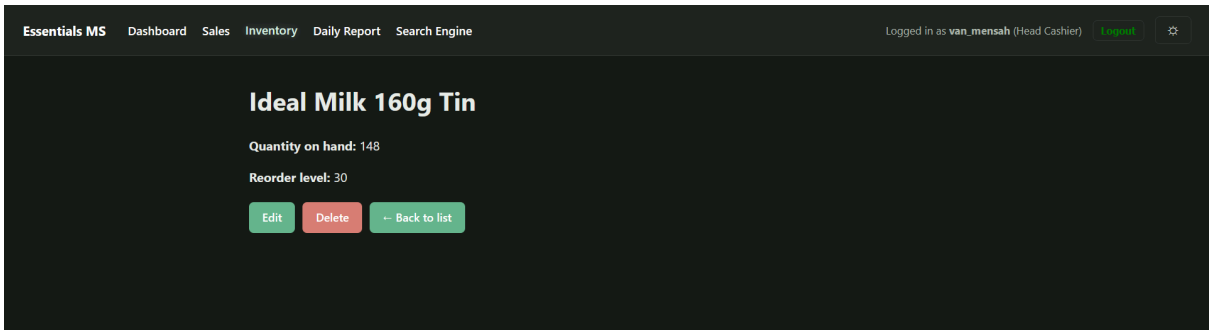


Fig 4.8

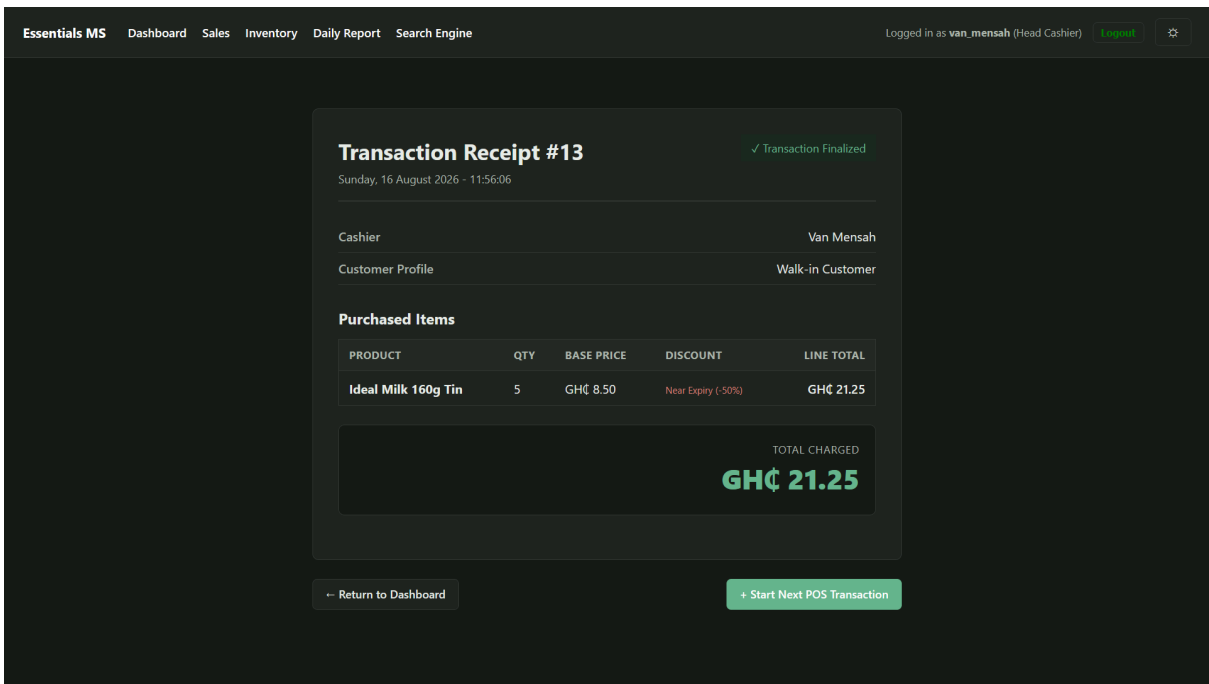


Fig 4.9

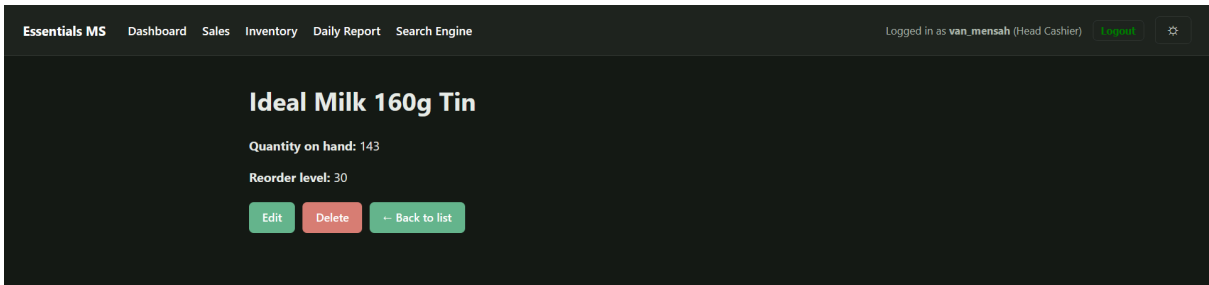


Fig 5.0

Automatically reduces stock by the amount purchased.

8. CHALLENGES AND SOLUTIONS:

Primary Key Naming Reversal

Problem: The initial schema design favoured Django's default surrogate id primary key on every model for simplicity, overriding the data dictionary's descriptive PK names (supplier_id, category_id, etc.). Once the database was actually created from the hand-written DDL (which used the descriptive names), this produced `OperationalError: Unknown column 'employee.id' in 'SELECT'` at runtime, because Django's models expected a column that did not exist in the physical table.

Solution: The decision was reversed. Every model now explicitly declares its primary key field to match the DDL's naming (e.g. `employee_id = models.AutoField(primary_key=True)`), and the hand-written SQL schema — not Django's migration history — was made authoritative. `python manage.py migrate store --fake` was used to mark Django's migration state as applied without re-running schema-altering SQL, since the tables already existed in their correct final form.

Django Version API Change: CheckConstraint

Problem: Early `CheckConstraint` definitions used the `check=` keyword argument, which raised a deprecation/removal error under the Django version in use.

Solution: All constraints were updated to the current `condition=` keyword argument, confirmed against the Django 6.1 documentation.

Trigger and Application-Layer Conflict

Problem: Two of the three triggers written for the SQL appendix (`trg_after_sale_item_insert`, `trg_after_sale_loyalty_update`) independently perform the same stock-deduction and loyalty-point-awarding logic that the Django checkout view already performs inside its

atomic transaction. Installing both on the live schema would silently double-apply both effects on every sale.

Solution: Only the read-only `trg_prevent_insufficient_stock` trigger (a `BEFORE INSERT` guard with no side effects) is installed on the live application database. The other two remain as documented SQL artifacts, demonstrating trigger competency for the SQL appendix without being deployed where they would corrupt live application data — with the reasoning kept explicit for the live demonstration and the report (Chapter 6.4).

Duplicate-Product Checkout Oversell

Problem: The original checkout implementation checked stock sufficiency per line item independently. A cart containing the same product across two separate lines (e.g. two lines of 6 units each against 10 in stock) could pass both individual checks while overselling by 2 units combined, since neither line's check accounted for the other.

Solution: Line items are now combined by product (summing quantities) before the stock sufficiency check runs, so the check always evaluates the true total being sold for each product, regardless of how many separate lines it was entered across.

9. CONCLUSION:

Summary:

The project delivers a normalized MySQL schema and a role-based Django web application covering the full operational cycle of a single supermarket: supplier and product management, purchasing, inventory tracking, checkout with genuine multi-table transactional consistency, two real business-rule discounts, and reporting driven directly from live data.

Achievements:

- A 3NF-normalized 12-table schema with explicit, individually-reasoned ON DELETE behaviour for every foreign key.
- A transactional checkout process using row-level locking and atomic rollback, tested both automatically and manually.
- Role-based access control built on Django's native Group/Permission system, enforced consistently across every view.
- A full SQL artifact set — views, stored procedures, user-defined functions, triggers, and ten advanced queries — with a considered, documented decision about which triggers are actually safe to run alongside the application.
- Two independently implemented layers of the same business intelligence: raw SQL (queries.sql) and Django ORM equivalents (the Advanced Analytics report), demonstrating the same reasoning expressed in both languages.

Limitations:

- Product expiry is tracked at the product level, not per delivery batch; two deliveries of the same product with different expiry dates cannot currently be distinguished.
- SaleItem.line_total reflects the per-line expiry discount but not the sale-level loyalty discount, so the sum of a sale's line totals does not exactly equal Sale.total_amount when the loyalty discount applies. This is a known, documented gap rather than an unnoticed inconsistency.
- The system is scoped to a single store; multi-branch operation was explicitly out of scope.

Future Improvements:

- Distribute the loyalty discount proportionally back across a sale's line items so stored line totals always reconcile exactly with the sale total.
- Move batch-level expiry tracking to purchase_order_item or a dedicated lot table, for physical accuracy when the same product is restocked with different expiry dates.
- Extend automated test coverage to the discount and expiry business rules directly, alongside the existing access-control and stock-deduction tests.

References

- Django Software Foundation. (n.d.-a). *Constraints reference*. Django.
<https://docs.djangoproject.com/en/stable/ref/models/constraints/>
- Django Software Foundation. (n.d.-b). *Database transactions*. Django.
<https://docs.djangoproject.com/en/stable/topics/db/transactions/>
- Django Software Foundation. (n.d.-c). *Django documentation*. Django.
<https://docs.djangoproject.com/en/stable/>
- Django Software Foundation. (n.d.-d). *Formsets*. Django.
<https://docs.djangoproject.com/en/stable/topics/forms/formsets/>
- Django Software Foundation. (n.d.-e). *Making queries*. Django.
<https://docs.djangoproject.com/en/stable/topics/db/queries/>
- Django Software Foundation. (n.d.-f). *Migrations*. Django.
<https://docs.djangoproject.com/en/stable/topics/migrations/>
- Django Software Foundation. (n.d.-g). *Models and databases*. Django.
<https://docs.djangoproject.com/en/stable/topics/db/>
- Django Software Foundation. (n.d.-h). *The Django admin site*. Django.
<https://docs.djangoproject.com/en/stable/ref/contrib/admin/>
- Django Software Foundation. (n.d.-i). *User authentication system*. Django.
<https://docs.djangoproject.com/en/stable/topics/auth/default/>
- Django Software Foundation. (n.d.-j). *Working with forms*. Django.
<https://docs.djangoproject.com/en/stable/topics/forms/>
- Faraglia, D. (n.d.). *django-envron documentation*. Read the Docs.
<https://django-envron.readthedocs.io/>
- MDN Web Docs. (n.d.). *CSS custom properties (--*)*. Mozilla.
https://developer.mozilla.org/en-US/docs/Web/CSS/--*

Oracle Corporation. (n.d.-a). *CHECK constraints*. MySQL 8.0 Reference Manual.

<https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html>

Oracle Corporation. (n.d.-b). *FOREIGN KEY constraints*. MySQL 8.0 Reference Manual.

<https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>

Oracle Corporation. (n.d.-c). *MySQL 8.0 reference manual*.

<https://dev.mysql.com/doc/refman/8.0/en/>

Oracle Corporation. (n.d.-d). *Triggers*. MySQL 8.0 Reference Manual.

<https://dev.mysql.com/doc/refman/8.0/en/triggers.html>

Oracle Corporation. (n.d.-e). *Window function concepts and syntax*. MySQL 8.0 Reference Manual.

<https://dev.mysql.com/doc/refman/8.0/en/window-functions-usage.html>

Python Software Foundation. (n.d.-a). *collections — Container datatypes*. Python documentation. <https://docs.python.org/3/library/collections.html>

Python Software Foundation. (n.d.-b). *decimal — Decimal fixed point and floating point arithmetic*. Python documentation.

<https://docs.python.org/3/library/decimal.html>

van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style guide for Python code*. Python.org. <https://peps.python.org/pep-0008/>